

## UNIVERSITY OF CLAUDE · COURSE SYLLABUS

# CSCE 658 — Randomized Algorithms

*A coin as an engineering primitive — and the concentration bounds that make it trustworthy*

## AT A GLANCE

**Credits:** 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 8 (with CSCE 627 and CSCE 637)

**Prerequisites:** CSCE 629 Analysis of Algorithms; probability through expectation, variance, and independence; CSCE 637 Module 08 for the complexity-class view; linear algebra for Modules 09–10

**Effort:** 12–14 hours per week · 13 modules + 2 project weeks

**Deliverable:** A randomised algorithm you implemented, with its failure probability derived and then measured against the derivation — plus one deterministic baseline it had to beat

## Course description

CSCE 627 asked what is possible and CSCE 637 asked what it costs. This course is about a technique — and it is the most immediately usable of the three, because **almost every system you have built already depends on it**. Hash tables, load balancers, caches, sampling, Bloom filters, Monte Carlo rendering, stochastic gradient descent: all randomised, and all analysed by the tools of Module 02.

**The organising idea is that a probabilistic guarantee is a guarantee. Module 02 is the whole course in one module:** Markov, Chebyshev, Chernoff, and the union bound, which between them bound the failure probability of almost every randomised algorithm anyone writes. **A failure probability of  $2^{-40}$  is below the probability that the hardware computed a deterministic answer correctly**, which is the argument that makes the whole subject respectable.

**The second theme is that randomness buys three distinct things and they are worth separating. It defeats an adversary** (hashing, quicksort's pivot), **it simplifies an algorithm** (skip lists against balanced trees), and **it allows sampling where exact computation is infeasible** (streaming, Monte Carlo integration). **Module 01 makes the distinction and the rest of the course keeps it.**

**The third is that randomness is also a proof technique. Module 07's probabilistic method proves objects exist by showing a random one works** — which produces existence results no constructive argument has matched, and then **Module 12 shows how to remove the randomness afterwards.**

**And the closing position is about honesty with the randomness itself. Module 13 is about seeds, reproducibility, the quality of your generator, and reporting a failure probability you have actually measured** — because a randomised algorithm whose randomness is not what you assumed has no

guarantee at all.

## Learning outcomes

---

On finishing this course you should be able to do the following without reference material.

- 1 Distinguish the three things randomness buys and choose accordingly.
- 2 Apply Markov, Chebyshev, Chernoff, and the union bound correctly.
- 3 Analyse balls-and-bins problems and the power of two choices.
- 4 Explain and implement randomised data structures.
- 5 Design a sketch for a streaming problem and bound its error.
- 6 Explain phase transitions in random graphs.
- 7 Use the probabilistic method to prove existence.
- 8 Apply randomised rounding to an approximation problem.
- 9 Bound a Markov chain's mixing time.
- 10 Apply dimension reduction with a stated distortion bound.
- 11 Explain where randomness is essential in distributed systems.
- 12 Derandomise an algorithm by conditional expectations or limited independence.
- 13 Report a randomised result with its failure probability measured.

## Module schedule

---

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

| Wk | Module                            | Core question it answers                                             |
|----|-----------------------------------|----------------------------------------------------------------------|
| 1  | <b>Why Randomness</b>             | What does randomness actually give you?                              |
| 2  | <b>Concentration</b>              | How do you bound the probability that a random variable misbehaves?  |
| 3  | <b>Balls and Bins</b>             | If you throw $n$ things into $n$ places at random, what happens?     |
| 4  | <b>Randomised Data Structures</b> | Why would you choose a structure that is only probably balanced?     |
| 5  | <b>Sketches and Streams</b>       | What can you compute in one pass with sublinear memory?              |
| 6  | <b>Random Graphs</b>              | What does a random graph look like, and why does it change suddenly? |
| 7  | <b>The Probabilistic Method</b>   | How can a probability argument prove an object exists?               |
| 8  | <b>Randomised Rounding</b>        | What do you do with an answer of 0.37?                               |

| Wk | Module                                   | Core question it answers                                               |
|----|------------------------------------------|------------------------------------------------------------------------|
| 9  | <b>Markov Chains and Mixing</b>          | How long must a random process run before its state is representative? |
| 10 | <b>Dimension and Sampling</b>            | Can you shrink high-dimensional data without losing what matters?      |
| 11 | <b>Randomness in Distributed Systems</b> | Where is randomness the only thing that works?                         |
| 12 | <b>Derandomisation</b>                   | Having used randomness to design the algorithm, can you remove it?     |
| 13 | <b>Using Randomness Honestly</b>         | What does a randomised result actually guarantee?                      |

## Projects

---

### Project 1 — Derive the bound, then measure it (after Module 7)

Implement several randomised algorithms, derive each one's failure probability, and measure whether the derivation was right.

#### Requirements

- **Four algorithms implemented** from Modules 03–05, including one data structure and one sketch.
- **A derived bound for each**, with the concentration inequality named and the steps shown.
- **An empirical failure rate for each**, over enough trials to resolve the bound.
- **A plot of empirical against predicted** for each, on a log scale.
- **One case where the bound is loose**, with an explanation of why.
- **A deterministic comparison** for at least two of the four.

#### Done means

- **Every bound derived rather than cited**, with the inequality named and the independence assumption stated.
- **The empirical rate consistent with the bound**, or an explanation of the discrepancy — both of which are acceptable and one of which is more interesting.
- **The loose case identified and explained** — usually Chebyshev where Chernoff applied, or a union bound over correlated events.
- **And the deterministic comparison honest**, including where the deterministic structure wins.

### Project 2 — A randomised solution to a real problem (after Module 12)

Solve a problem you care about with randomness, and establish what the guarantee actually is.

#### Requirements

- **A real problem**, with the deterministic approach implemented first and measured.
- **A randomised algorithm**, with the failure probability or approximation quality derived.
- **The seeds fixed and reported**, and results across at least ten seeds.
- **A sensitivity test on the random source**: run it with a weak generator and report what changes.
- **A derandomisation attempt** (Module 12), or an argument that none is practical.
- **The honest claim**, in Module 13's form.

#### Done means

- **The deterministic baseline measured, not assumed** — and it may win, which is a complete result.
- **The failure probability derived and measured**, and the two compared.

- **The weak-generator test performed**, because it is the one thing nobody does and it is where real failures come from.
- **And a statement of what the algorithm guarantees, scoped to the randomness assumption** — which is the graded part.

## Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

| Component        | Weight | Standard                                                                                        |
|------------------|--------|-------------------------------------------------------------------------------------------------|
| Module exercises | 40%    | Code runs and produces the specified output. Not 'I understand how I would do it.'              |
| Projects         | 45%    | In the portfolio repository, with a README showing results and a written note on what was hard. |
| Self-checks      | 15%    | Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.  |

## Required materials

All free. Nothing in this course requires a purchase.

| Resource                                                      | Role                                                                                                                                                          |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Motwani & Raghavan — Randomized Algorithms                    | <b>The primary source and still the standard.</b> Modules 02–09 follow it. Library copy; the authors' lecture notes from the courses it grew out of are free. |
| Mitzenmacher & Upfal — Probability and Computing, 2nd edition | <b>The better first book</b> , and stronger on Modules 02, 03, and 09. Library copy; the authors' course pages carry free problem sets.                       |
| Alon & Spencer — The Probabilistic Method                     | <b>Module 07's reference</b> , and the book that made the method a subject. Library copy; Spencer's free lecture notes cover the core.                        |
| Cormode & Yi — Small Summaries for Big Data (free)            | <b>Module 05's reference, free in full</b> — sketches and streaming algorithms with their error bounds, written for practitioners.                            |
| Levin & Peres — Markov Chains and Mixing Times (free PDF)     | <b>Module 09's reference, free from the authors</b> — the definitive treatment of mixing, and chapters 1–7 are what this course needs.                        |
| Vadhan — Pseudorandomness (free)                              | <b>Module 12's reference, free in full</b> — expanders, extractors, and generators, which is where derandomisation actually lives.                            |

## Tooling

---

- **A language with a decent generator and the ability to seed it explicitly.** Module 13 requires every experiment to be reproducible, so build seeding in from the first line.
- **NumPy, or equivalent.** Most of this course's experiments are vectorised sampling, and a loop-based implementation will make the large-n experiments painful.
- **A plotting setup for empirical distributions.** The central activity of this course is deriving a bound and then measuring whether it holds, and that is a histogram against a curve.
- **A proper statistical random source available** — `secrets` or `/dev/urandom` — **so you can compare it against a weak generator and see the guarantee fail** (Module 13 §2).
- **A streaming data source** for Module 05: a log file, a packet capture, or a generated stream too large for memory.
- **And a deterministic baseline for everything.** Every randomised algorithm in this course has a deterministic rival, and the comparison is the point.

## Academic honesty, restated for a solo student

---

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.